

EquipFlow — System Architecture Document

Version: 1.0.0
Last Updated: September 09, 2026
Variant: D5T3 (Industrial Field Maintenance + Cost Governor)

1. Purpose & Scope

This document defines the **authoritative system architecture** for EquipFlow. It serves as the single source of truth for all engineering decisions during MVP development and bridges the gap between:

- **Phase 1 — Project Definition** (business vision)
- **Phase 2 — Requirements Engineering v1.1** (functional baseline)
- **ITI Instructor Task** (mandatory engineering constraints)

All subsequent ADRs, designs, and implementation artifacts must trace back to this document.

2. Architectural Goals & Principles

2.1 Primary Goals

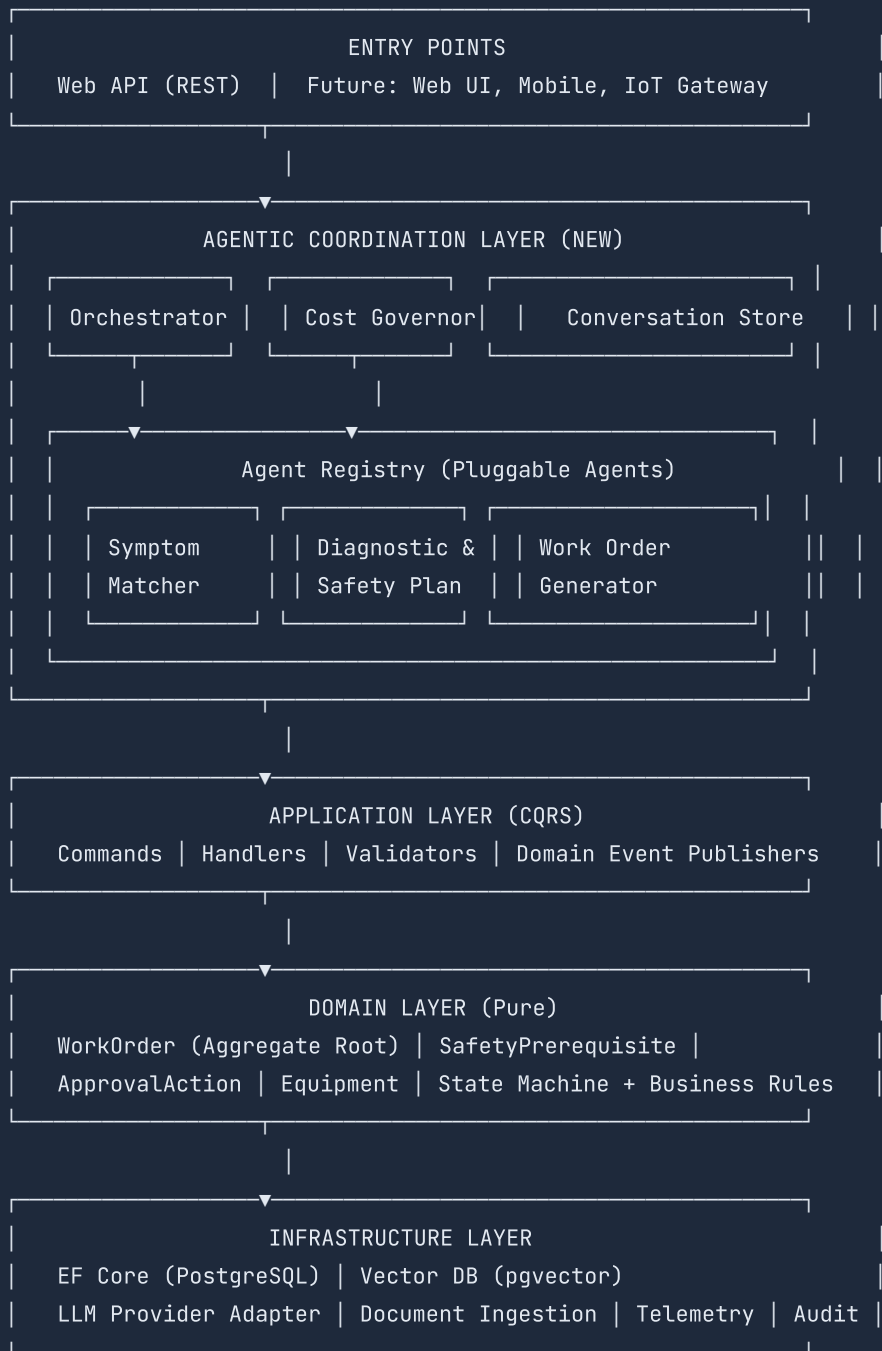
1. **Correctness First** — Domain model (Work Order state machine, safety prerequisites, approval gate) is strictly enforced regardless of AI involvement.
2. **Controlled AI Autonomy** — AI agents operate within a bounded, auditable envelope. No "runaway" agent behavior.
3. **Cost Governance (T3)** — Every LLM interaction and tool invocation is budget-aware and traceable.
4. **Retrieval Grounding** — No hallucinated maintenance advice; all technical claims cite equipment manuals.
5. **Deterministic Where Possible** — AI is used selectively; approval gates, safety checks, and write operations are deterministic.

2.2 Architectural Principles

Principle	Meaning
Clean Architecture	Domain is pure; no framework/infrastructure pollution
CQRS at Application Layer	Commands (writes) and queries (reads) are separate paths
Provider Abstraction	LLM provider is swappable; no direct vendor coupling
Explicit Boundaries	Every cross-layer interaction is via a defined port/interface
Observable by Default	Every agent run, tool call, and approval gate emits telemetry
Fail-Safe AI	On any AI failure, system defaults to human-escalation, not guesswork

3. High-Level Architecture

EquipFlow uses a **layered Clean/Hexagonal architecture** with a dedicated **Agentic Coordination layer** on top of the traditional application stack.



4. Component Specifications

4.1 Web API Layer

- **Tech:** ASP.NET Core 8/9 Minimal APIs
- **Key Endpoints (MVP):**
 - `POST /api/workorders` — create draft
 - `POST /api/workorders/{id}/safety` — add safety prerequisite
 - `POST /api/workorders/{id}/submit` — submit for supervisor approval
 - `POST /api/workorders/{id}/approve|reject` — supervisor decision (role=Supervisor required)
 - `POST /api/workorders/{id}/dispatch` — authorise dispatch (gated write tool)
 - `POST /api/chat` — conversational AI entry point (orchestrator)

4.2 Agentic Coordination Layer

4.2.1 Orchestrator

- Routes user intent to appropriate agent(s).
- Maintains conversation state across multi-turn interactions.
- Decides handoff sequence: `SymptomMatcher` → `Diagnostic & Safety Planner` → `Work Order Generator` .
- Emits `AgentRunStarted` / `AgentRunCompleted` telemetry.

4.2.2 Agent Registry — MVP Agents

Agent	Role	Tools	Input	Output
SymptomMatcher	Match symptoms to historical faults	<code>SearchManuals</code> , <code>QueryFaultHistory</code>	Symptom + equipment context	Ranked list of probable faults
Diagnostic & Safety Planner	Generate diagnostic steps + safety checklist	<code>GetEquipmentSpecs</code> , <code>GenerateSafetyChecklist</code>	Matched symptoms	Diagnostic plan + SafetyPrerequisites
Work Order Generator	Compose draft WorkOrder from plan	<code>CreateWorkOrder</code> , <code>ValidateBudget</code>	Diagnostic plan	Draft WorkOrder ID (or refusal)

Each agent:

- Receives a **scoped budget token** from Cost Governor.
- Can invoke **only** tools declared in its spec.
- Returns a **structured result** (not free-form text).

4.2.3 Cost Governor (T3 Twist)

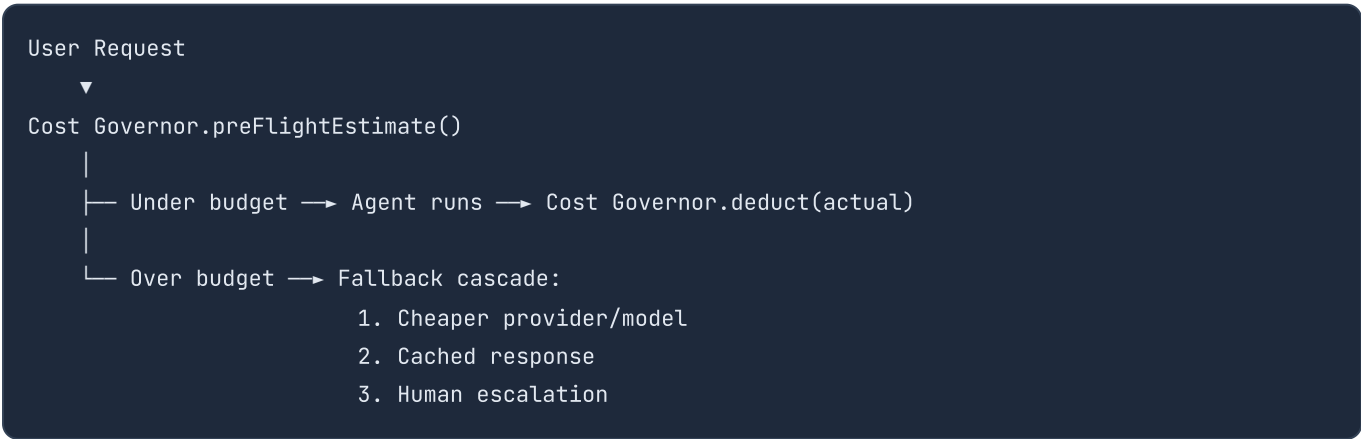
Responsibilities:

1. Maintain per-user budgets (stored in DB, refreshed per session).
2. **Pre-flight estimate** cost of every agent run and tool call.
3. Block/downgrade execution when budget exhausted.
4. Deduct actual cost after execution.
5. Emit `BudgetDepleted` events.

Budget Model:

- Unit: **USD-equivalent tokens** (normalized across providers).
- Granularity: Per-user daily budget, configurable per role.
- Pre-flight: `estimated_tokens × provider_rate × safety_margin(1.2)` .

Enforcement Flow:



4.2.4 Conversation Store

- PostgreSQL table `ConversationTurns` with JSON payload.
- Used for audit, evaluation, debugging.

4.3 Application Layer (CQRS)

New Commands Required:

- `AgentCreateWorkOrderCommand` (gated by Work Order Generator)
- `AgentSuggestSafetyPrerequisitesCommand`

New Ports Required:

- `IEquipmentRepository`
- `IManualSearchPort` (RAG)
- `IFaultHistoryPort`

4.4 Domain Layer

No structural changes needed. Current state machine correctly enforces:

- `Draft → PendingApproval → Approved/Rejected → Dispatched`
- Mandatory safety prerequisites before submit
- Only Supervisor can approve
- Only Approved WOs can be dispatched

4.5 Infrastructure Layer Extensions

Component	Tech	Purpose
Relational DB	PostgreSQL 16	WorkOrders, Budgets, Users, Audit
Vector DB	pgvector	Equipment manual embeddings
LLM Provider	Abstracted via <code>ILLMProvider</code>	Pluggable (OpenAI/Anthropic/Local)
Telemetry	OpenTelemetry → OTLP	Traces, metrics, logs

5. Key Flows

5.1 End-to-End AI-Assisted Work Order Creation

```
Technician: "CNC machine X-200 making grinding noise"
  ▼
[Web API] POST /api/chat
  ▼
[Orchestrator] Intent → "diagnose fault"
  ▼
[Cost Governor] pre-flight ✓
  ▼
[SymptomMatcher Agent]
  ├── SearchManuals("X-200 grinding")
  ├── QueryFaultHistory("X-200")
  └── Returns: ["bearing wear (82%)", "gearbox (15%)"]
  ▼
[Diagnostic & Safety Planner]
  ├── GetEquipmentSpecs("X-200")
  ├── GenerateSafetyChecklist("bearing")
  └── Returns: DiagnosticPlan + 4 SafetyPrerequisites
  ▼
[Work Order Generator]
  ├── ValidateBudget($2450) → OK
  ├── CreateWorkOrder(plan) → Draft WO#ABC-123
  └── Returns: WorkOrderId + summary
  ▼
[Cost Governor] deducts actual tokens
  ▼
Response: "Draft WO#ABC-123 created. Please review and submit."
```

5.2 Supervisor Approval Gate (Deterministic)

```
POST /api/workorders/ABC-123/submit
  ▼
[SubmitWorkOrderForApprovalHandler]
  ├── Load WorkOrder
  ├── WorkOrder.SubmitForApproval(userId)
  │   ├── Validate: all mandatory SafetyPrerequisites done ✓
  │   └── Transition → PendingApproval
  ├── Persist
  └── Publish WorkOrderSubmittedEvent
```

5.3 Dispatch Authorization (Gated Write Tool)

```
POST /api/workorders/ABC-123/dispatch (role=Supervisor verified)
  ▼
[DispatchHandler]
  ├── WorkOrder.MarkDispatched() [Approved → Dispatched]
  ├── Persist + audit
  └── Trigger downstream scheduling
```

5.4 Cost Governor — Budget Exhaustion

```
User request → Cost Governor pre-flight = $0.042
|
└─ User budget remaining = $0.015 (OVER)
    ▼
    Fallback cascade:
    1. Try cheaper model (GPT-4o-mini) → re-estimate
    2. Try cached semantic match
    3. Structured refusal:
        {
            "status": "budget_exhausted",
            "remaining": "$0.015",
            "estimated_cost": "$0.042",
            "options": ["wait_for_reset", "escalate_to_supervisor"]
        }
```

6. Technology Stack

Layer	Technology	Version	Justification
Runtime	.NET 10 LTS	10.x	Latest LTS (Nov 2025), supported until Nov 2028
Web Framework	ASP.NET Core Minimal APIs	10.x	Ships with .NET 10; lightweight, testable
ORM	EF Core	10.x	Ships with .NET 10; already configured patterns transfer
DB	PostgreSQL	18	Latest major (Sep 2025); production-grade, JSON, pgvector
Vector Search	pgvector	latest	Co-located with relational data
LLM Abstraction	Custom ILLMProvider	—	Provider Abstraction requirement
Telemetry	OpenTelemetry .NET	latest	Vendor-neutral standard
Testing	xUnit + NSubstitute + Testcontainers	latest	Matches existing test patterns

7. Cross-Cutting Concerns

7.1 Observability

- **Traces:** `work_order.lifecycle`, `agent.run`, `tool.invocation`, `rag.retrieval`
- **Metrics:** `tokens.consumed`, `budget.remaining`, `agent.success_rate`
- **Logs:** Structured JSON via Serilog, correlated by `TraceId`

7.2 Audit Log

Immutable append-only table capturing:

- Every `ApprovalAction` (already in domain)
- Every agent invocation (`agent_id`, `input_hash`, `output_hash`, `tokens`, `cost`)
- Every tool call (`name`, `parameters`, `result_summary`)
- Every budget deduction

7.3 Security

- **Auth:** JWT bearer tokens
- **AuthZ:** Role-based + resource-based policies
- **AI Guardrails:**
 - Prompt injection defense
 - Output filtering (no PII, no unsafe advice)
 - Tool sandboxing (agents never bypass Approval Gate)

8. Deployment Model



Production: K8s + managed Postgres (RDS) + managed observability.

9. Traceability Matrix

Requirement (Phase 2 v1.1)	Location	Status
≥3 agents + orchestrator	\$4.2	Designed
Supervisor approval before dispatch	\$4.4, \$5.2	Implemented
Write tool gated by approval	\$5.3	Implemented
Cost Governor	\$4.2.3	Designed
Pre-flight estimation	\$4.2.3	Designed
Provider Abstraction	\$6 (ILLMProvider)	Designed
Observability	\$7.1	Designed

10. Known Risks & Mitigations

Risk	Mitigation
Agent hallucination → unsafe advice	RAG grounding + output filtering + mandatory human review
Cost Governor false-positives	Configurable safety margin + supervisor override
Multi-agent latency	Parallel retrieval; cache equipment specs
Vector retrieval misses relevant sections	Hybrid search (vector + BM25)
Agent circumvents Approval Gate	Tools exposed only via Application-layer commands

11. Out of MVP Scope

- Streaming chat responses
 - Proactive fault prediction (IoT)
 - Multi-tenancy
 - Mobile offline mode
 - Fine-tuned domain model
-

12. References

1. [docs/references/EquipFlow_Phase_1_Project_Definition_FINAL.pdf](#)
 2. [docs/references/EquipFlow_Phase_2_Requirements_Engineering_FINAL_v1.1.pdf](#)
 3. [docs/references/ITI-Instructor-Task.pdf](#)
-